

Segmentazione a Soglia (*Thresholding*)

Nella **segmentazione a soglia**, ogni pixel viene associato a un tipo di tessuto basandosi unicamente sulla sua **intensità di segnale**. Questo processo equivale a scegliere **una o più soglie** nell'istogramma dell'immagine per classificare i pixel.

Meccanismo e Applicazione

Considerando l'immagine biomedica e il suo istogramma, la segmentazione a soglia opera come segue:

1. **Identificazione delle Classi:** Si identificano i picchi nell'istogramma, dove ciascun picco descrive la distribuzione del segnale associato a una specifica classe di tessuto. Il numero di soglie necessarie è $N - 1$, dove N è il numero di classi da distinguere.
2. **Scelta delle Soglie:** Fissando le soglie $T_1, T_2, \dots, T_{N-1}$, l'immagine viene suddivisa. Ad esempio, per tre classi (sfondo, acqua, olio) e due soglie ($T_1 = 90, T_2 = 500$):
 - **Classe 1 (Sfondo):** $s < T_1$
 - **Classe 2 (Acqua):** $T_1 \leq s < T_2$
 - **Classe 3 (Olio):** $s \geq T_2$
3. **Creazione delle Maschere:** Per ogni classe, si crea una maschera (*mask*) binaria in cui i pixel che soddisfano la condizione di soglia sono posti a un valore non nullo, e gli altri a zero.

!Pasted image 20251129202220.png

Limiti Fondamentali della Segmentazione a Soglia

L'esempio pratico evidenzia i limiti intrinseci di questo metodo:

1. **Impossibilità di Distinguere per Topologia:** La segmentazione a soglia **non è in grado di distinguere oggetti topologicamente diversi** ma ai quali è associato lo stesso livello di segnale.
 - **Problema Clinico:** Nell'esempio, i *pattern* 1-3-6 e 2-6 non sono distinguibili. Questo è critico in applicazioni cliniche dove due strutture vicine hanno la stessa intensità (es. ventricolo destro e sinistro nel cuore, grasso viscerale e grasso sottocutaneo). Questo è un **limite intrinseco** del metodo.
2. **Pixel Spuri (Rumore):** Possono apparire pixel spuri o assegnazioni erranee. Questo è dovuto all'**imperfetta separazione dei picchi** nell'istogramma, spesso indotta dalle variazioni di segnale generate dal **rumore**. La qualità dipende dal **Contrasto-Rumore (CNR)** tra i tessuti da separare.
3. **Definizione Manuale della Soglia:** Le soglie vengono spesso definite **manualmente** tramite esame visivo dell'istogramma.

Soluzioni e Approcci Correttivi

I limiti della segmentazione a soglia possono essere mitigati combinando questo approccio con algoritmi successivi:

- **Superamento del Limite Topologico (1):** Questo limite può essere superato applicando algoritmi di tipo **topologico** alle maschere prodotte, come l'algoritmo di **labeling** (*connected components labeling*), che è in grado di distinguere oggetti spazialmente separati anche se appartengono alla stessa classe di intensità.
 - **Correzione degli Errori di Rumore (2):** Gli errori di segmentazione indotti dal rumore possono essere corretti applicando algoritmi di **filtraggio** al risultato della segmentazione.
 - **Ricerca Automatica della Soglia (3):** Esistono vari approcci automatici che consentono di trovare la soglia "ottima" che divide i pixel in classi. Tali algoritmi adottano un approccio di **apprendimento non supervisionato** (*unsupervised learning*).
-

Algoritmo di Otsu

L'algoritmo di **Otsu** è un metodo di segmentazione a soglia non supervisionato, automatico, che trova la soglia ottimale nell'istogramma di un'immagine.

Principio di Ottimizzazione

Nella sua versione originale, l'algoritmo di Otsu presume che nell'immagine da segmentare siano presenti **due sole classi** (background e foreground). Successivamente, può essere esteso a più classi (*multi Otsu method*).

L'algoritmo cerca di trovare la soglia t che separi l'immagine in due sottoclassi che siano il più possibile omogenee al loro interno.

Varianza Intra-Classe e Inter-Classe L'algoritmo standard di Otsu calcola la soglia ottima **minimizzando la varianza intra-classe** (σ_W^2):

$$\sigma_W^2(t) = \omega_1(t)\sigma_1^2(t) + \omega_2(t)\sigma_2^2(t)$$

dove: - $\omega_i(t)$ è la probabilità (o peso) di una classe separata dall'altra dalla soglia t . - $\sigma_i^2(t)$ è la varianza (SD al quadrato) della classe i .

Si dimostra che **minimizzare la varianza intra-classe è equivalente a massimizzare la varianza inter-classe** (σ_B^2):

$$\sigma_B^2(t) = \omega_1(t)(\mu_1(t) - \mu_T)^2 + \omega_2(t)(\mu_2(t) - \mu_T)^2$$

dove: - $\mu_i(t)$ è il valor medio di una classe. - μ_T è il valore medio globale dell'immagine.

Intuitivamente, immaginando un istogramma a due picchi, il metodo di Otsu consiste nel trovare la soglia t intermedia tra i due picchi che li divida in modo ottimale.

Implementazione Algoritmica (Ricerca Esaustiva)

Il metodo di Otsu opera secondo un processo di **ottimizzazione basato sulla ricerca esaustiva** su tutti i possibili valori del parametro t .

Passaggi Algoritmici:

1. **Calcolo dell'Istogramma:** Si calcola l'istogramma h dell'immagine. La probabilità $p(i)$ di un livello di grigio i si ottiene normalizzando l'istogramma per il numero totale di pixel dell'immagine.
2. **Calcolo della Varianza per Ogni t :** Per ogni soglia t (da 0 al massimo livello di grigio) si calcolano i pesi e le medie delle due classi:

- Classe 1 (inferiore o uguale a t):

$$\omega_1(t) = \sum_{i=0}^t p(i) \quad \mu_1(t) = \frac{\sum_{i=0}^t i \cdot p(i)}{\omega_1(t)}$$

- **Classe 2 (superiore a t):** $\omega_2(t)$ e $\mu_2(t)$ si calcolano in modo analogo operando le somme da $t + 1$ in avanti. Si calcola poi $\sigma_B^2(t)$.

3. **Selezione della Soglia Ottimale:** Si sceglie il t che **massimizza** $\sigma_B^2(t)$.

In questo modo, la soglia t viene definita in modo **automatico**.

Efficienza e Considerazioni Finali

- **Efficienza Computazionale:** Il calcolo della quantità $\sigma_B^2(t)$ è molto rapido, permettendo al calcolo di t di essere effettuato in **tempi ragionevoli**.
- **Dipendenza dalla Profondità:** Il tempo di calcolo è fortemente dipendente dalla **profondità dell'immagine** (numero di livelli di grigio).
- **Soglie Multiple:** Possono esistere più valori di t per cui $\sigma_B^2(t)$ è massima. In questi casi, l'algoritmo restituisce il **valore medio** tra i valori di t che massimizzano $\sigma_B^2(t)$.

In MATLAB, il metodo di Otsu è implementato nella funzione **otsuthresh** (quando si utilizza l'istogramma) o **graythresh** (quando si opera direttamente sull'immagine).

Introduzione al Clustering

Il **Clustering** è una tecnica di analisi dei dati volta alla selezione e raggruppamento di elementi omogenei in un insieme di dati. L'approccio è del tipo **unsupervised**, nel senso che l'algoritmo di clustering riesce a dividere i dati in una serie di insiemi avendo come unico input il numero di insiemi da trovare.

Nel campo dell'analisi delle immagini biomediche: - Le **righe** della tabella rappresentano le locazioni spaziali (pixel/voxel) - Le **colonne** rappresentano i valori di segnale acquisiti

!Pasted image 20251129202811.png!Pasted image 20251129202820.png

Classificazione degli Algoritmi di Clustering

1. **Clustering esclusivo (K-means):** Un dato deve appartenere ad uno ed un solo cluster
2. **Clustering non esclusivo (Fuzzy C-Means):** Un dato può appartenere a più cluster con diversi gradi di appartenenza
3. **Clustering probabilistico (EM):** Basato su modelli probabilistici delle distribuzioni

Algoritmo K-means per il Clustering

L'algoritmo **K-means** (MacQueen, 1967) è un algoritmo base di **apprendimento non supervisionato** utilizzato per la classificazione dei dati.

Funzione Obiettivo e Complessità

Dato un insieme di dati $Y = \{y_1, \dots, y_n\}$ da classificare in C gruppi (cluster) e una distanza $\|\cdot\|$ definita su Y , l'algoritmo mira a minimizzare la seguente **funzione obiettivo** (J):

$$J = \sum_{j \in \Omega} \sum_{k=1}^C \|y_{jk} - v_k\|^2$$

dove: - y_{jk} è un dato y_j assegnato al cluster k - v_k è il **centroide** del cluster k

La funzione J è un indicatore della distanza complessiva dei dati dai centri dei rispettivi cluster.

- **Complessità:** Il problema di trovare l'assegnazione che realizza il minimo assoluto di J è **NP-completo**, con complessità computazionale $\mathcal{O}(n^{dC+1} \log n)$.

Procedura Iterativa K-means

L'algoritmo K-means è un ottimizzatore locale e richiede la definizione iniziale del numero di cluster C .

Passaggi Principali:

1. **Inizializzazione:** Assegnare un valore iniziale ai C centroidi v_k (in modo casuale, ma sufficientemente lontani tra loro).
2. **Iterazione** (Ripetuta finché i v_k si stabilizzano):
 - **Assegnazione:** Assegnare ogni dato y a uno e un solo cluster sulla base della distanza minima dal centroide.
 - **Aggiornamento:** Aggiornare i valori dei centroidi v_k calcolandoli come il centro di massa dei dati appartenenti al cluster k .

L'algoritmo **converge sempre**, ma la configurazione finale non è garantita essere il **minimo assoluto**.

Dipendenza dalle Condizioni Iniziali Poiché l'algoritmo K-means è un **ottimizzatore locale**, lo stato di convergenza dipende dalla definizione iniziale dei centroidi.

- **Strategia di Ottimizzazione:** Ripetere il procedimento più volte (**nreplicates** in MATLAB) assegnando valori iniziali casuali e scegliendo la configurazione finale che minimizza il valore di J .

Aspetti Implementativi e Difficoltà

L'algoritmo K-means è implementato in MATLAB nella funzione **kmeans**. Difficoltà operative:

- **Inizializzazione dei Centroidi:** Necessario un metodo di inizializzazione (opzione **start**)
- **Dipendenza dal Risultato Iniziale:** Risolvibile con opzione **nreplicates**

- **Svuotamento dei Cluster:** Gestito dall'opzione `EmptyAction`
- **Scelta della Metrica:** Opzione `Distance`
- **Numero di Cluster (C):** Parametro di input predefinito

Varianti Una variante del K-means è l'algoritmo **ISODATA** (*Iterative Self-Organizing Data Analysis Technique*), dove il numero di cluster C **non è un parametro fisso** ma viene computato dinamicamente.

Applicazione alla Segmentazione di Immagini

Nel *clustering* di immagini, la metrica utilizzata è tipicamente la **differenza assoluta** o la **media quadratica delle differenze** dei livelli di grigio.

!Pasted image 20251129202948.png !Pasted image 20251129203004.png !Pasted image 20251129203020.png

Clustering Non Esclusivo (FCM)

Nel **clustering non esclusivo** (*fuzzy clustering*), un dato può appartenere a **più cluster** contemporaneamente, con diversi **livelli di appartenenza**. La somma dei livelli di appartenenza per un dato su tutti i possibili cluster deve essere 1.

Fondamenti Teorici: Logica Fuzzy

L'algoritmo **Fuzzy C-Means (FCM)** generalizza il K-means introducendo la **Logica Fuzzy** (o Logica Sfumata), ideata da Lotfi Zadeh.

- **Grado di Verità:** La Logica Fuzzy si basa su un **grado di verità** o **valore di appartenenza** (u) che può assumere valori **intermedi** tra 0 e 1.
- **Contrasto con la Probabilità:** Il concetto di appartenenza *fuzzy* non ha nulla a che vedere con la probabilità. Nella probabilità un'affermazione è vera o falsa con una certa probabilità; nella logica *fuzzy* è **insieme vera e falsa** con un certo grado.

Funzione Obiettivo e Parametro Fuzzyness

L'algoritmo FCM (Dunn, 1973; Bezdek, 1981) minimizza:

$$J_{FCM} = \sum_{j \in \Omega} \sum_{k=1}^C u_{jk}^m \|y_j - v_k\|^2$$

dove: - u_{jk} è il **grado di appartenenza** di y_j al cluster k - m è il parametro **fuzzyness**, tipicamente $m = 2$

Se u è una matrice binaria, la funzione J_{FCM} diviene uguale alla funzione $J_{K-means}$.

Aggiornamento Iterativo

Le formule di aggiornamento sono:

$$u_{j,k} = \frac{1}{\sum_{q=1}^C \left(\frac{\|y_j - v_k\|}{\|y_j - v_q\|} \right)^{\frac{2}{m-1}}}$$

$$v_k = \frac{\sum_{j=1}^N u_{jk}^m y_j}{\sum_{j=1}^N u_{jk}^m}$$

L'algoritmo è implementato in MATLAB nella funzione **fcm**.

Rilevanza Clinica: Gestione del PVE

L'approccio FCM è di grande interesse nella segmentazione delle immagini mediche perché consente di tenere conto dell'**Effetto Volume Parziale (PVE)**.

L'approccio FCM consente di assegnare a pixel con segnale misto un **valore di appartenenza distribuito** tra i tessuti presenti, **interpretando in modo corretto il PVE**.

Algoritmo EM (Expectation Maximization)

L'**Algoritmo EM** introduce il **clustering basato su modelli** per superare i limiti degli algoritmi tradizionali, includendo informazioni sulla distribuzione di probabilità dei vari cluster.

L'Approccio Basato su Modelli

La distribuzione dei dati all'interno dei singoli cluster viene modellata utilizzando **funzioni probabilistiche note**. L'insieme dei dati totale è modellato come una **combinazione lineare** di queste distribuzioni componenti:

$$h(s) = \sum_{k=1}^K P_k G(s|M_k, \sigma_k)$$

Dove: - $h(s)$ è l'istogramma - K è il numero delle classi - P_k è la probabilità "a priori" della classe k - $G(s|M_k, \sigma_k)$ è la distribuzione gaussiana per la classe k

Funzionamento dell'Algoritmo EM

Il procedimento iterativo si compone di due passi principali:

Passo E (Expectation) Calcolo della **probabilità che il dato x_j sia stato generato dalla gaussiana i** :

$$p_{ij} = P_i G(x_j|M_i, \sigma_i)$$

Passo M (Maximization) Aggiornamento dei parametri delle gaussiane:

$$\mu_i \leftarrow \frac{1}{p_i} \sum_j p_{ij} x_j$$

$$\sigma_i \leftarrow \frac{1}{p_i} \sum_j p_{ij} (x_j - \mu_i)^2$$

$$P_i \leftarrow \frac{p_i}{\sum_k p_k}$$

L'algoritmo EM-GM è implementato in Matlab dalla funzione `fitgmdist`.

Clustering Gerarchico

Il clustering gerarchico costruisce una struttura ad albero (dendrogramma) che rappresenta le relazioni di similarità tra i dati.

Esempio Pratico: Single-Linkage

Consideriamo cinque punti dati P_1, P_2, P_3, P_4, P_5 con matrice delle distanze:

	P1	P2	P3	P4	P5
P_1	0	2.0	6.0	7.0	9.0
P_2	2.0	0	5.0	6.5	8.0
P_3	6.0	5.0	0	2.5	3.5
P_4	7.0	6.5	2.5	0	1.0
P_5	9.0	8.0	3.5	1.0	0

Iterazioni: 1. $L(1) = 1.0$: Fusione (P_4, P_5) 2. $L(2) = 2.0$: Fusione (P_1, P_2) 3. $L(3) = 2.5$: Fusione $(P_3, (P_4, P_5))$ 4. $L(4) = 5.0$: Fusione finale

Il metodo **Single-Linkage** tende a formare “catene” di punti, in quanto è sufficiente che due membri dei cluster siano vicini per innescare la fusione.

Metriche negli Algoritmi di Clustering

La scelta della **metrica** è cruciale, poiché definisce come viene misurata la **differenza (o distanza)** tra due punti.

La Famiglia delle Distanze di Minkowski

$$d(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{q=1}^K |x_{iq} - x_{jq}|^p \right)^{\frac{1}{p}}$$

Casi Particolari

- **Distanza Euclidea** ($p = 2$): Distanza in linea retta
- **Distanza Manhattan** ($p = 1$): Somma delle differenze assolute
- **Distanza di Chebyshev** ($p \rightarrow \infty$): Massima differenza tra le coordinate

Altre Metriche Comuni

Metrica	Applicazione Tipica
Correlazione	Segmentazione di immagini 2D+T (curve temporali)
Distanza di Mahalanobis	Variabili con scale diverse o correlate
Distanza di Jaccard	Dati binari o set di attributi

Importanza della Normalizzazione

Se i dati non sono omogenei, è opportuno applicare la **standardizzazione delle variabili** (Z-Score):

$$z = \frac{x - \mu}{\sigma}$$

In MATLAB: funzioni `zscore` o `normalize`.

!Pasted image 20251129203740.png
